

**UNIVERSIDAD NACIONAL AUTÓNOMA DE NICARAGUA
(UNAN-LEÓN)**

FACULTAD DE CIENCIAS Y TECNOLOGÍA



CARRERA: Ingeniería en tecnologías de la información con énfasis en desarrollo web y marketing digital.

COMPONENTE: App para web en ingles/español.

TRABAJO: Informe final del Sistema Web de Gestión de Inventario.

INTEGRANTES: Julio César Zapata Aguilar. Cod.unico: 65d265d5

Gustavo José López Romero Cod.unico: 6FA788e8

Cristóbal Inocente Ramos Cod.unico: 6b0c9339

DOCENTE: Aldo Berrios.

¡A la libertad por la universidad!

INFORME FINAL DEL PROYECTO

SISTEMA WEB DE GESTIÓN DE INVENTARIO

1. INTRODUCCIÓN

El presente informe documenta el desarrollo de un **Sistema Web de Gestión de Inventario**, construido mediante una arquitectura que integra un backend desarrollado con Laravel y una interfaz frontend tipo SPA desarrollada con React.

El sistema fue desarrollado con el propósito de gestionar información relacionada con **categorías y productos**, permitiendo realizar operaciones de creación, consulta, edición y eliminación, además de establecer una relación entre los productos y sus respectivas categorías.

Durante el desarrollo se implementaron mecanismos de autenticación, autorización mediante roles y permisos, validación de datos, comunicación mediante una API, manejo de errores HTTP y navegación mediante rutas protegidas en React.

La aplicación utiliza Laravel como responsable de la lógica del servidor, seguridad, validación y acceso a la base de datos, mientras que React se encarga de la interfaz de usuario y de la interacción con la API.

El desarrollo también contempló la organización del frontend mediante componentes, rutas, formularios controlados y un servicio centralizado para las solicitudes HTTP.

Finalmente, se realizaron las pruebas establecidas en la guía del proyecto, identificadas como P1 a P12, además de dos pruebas adicionales. La evidencia correspondiente será incorporada mediante las capturas de pantalla obtenidas durante las pruebas.

2. DESCRIPCIÓN Y PROBLEMÁTICA DEL PROYECTO

2.1 Descripción del proyecto

El proyecto consiste en una aplicación web para la gestión de un inventario. El sistema permite administrar principalmente dos entidades:

- Categorías.
- Productos.

Las categorías permiten organizar los productos, mientras que cada producto almacena información como:

- Categoría.
- Nombre.
- Código.
- Descripción.
- Precio.
- Stock.
- Stock mínimo.
- Imagen, cuando corresponde.

El sistema permite crear, consultar, modificar y eliminar registros mediante una interfaz desarrollada en React que se comunica con una API desarrollada en Laravel.

Además, se implementó un sistema de autenticación y autorización para controlar el acceso a las diferentes funcionalidades.

2.2 Problemática

La gestión de productos y categorías requiere mantener información organizada y consistente. Cuando estas operaciones no están centralizadas en un sistema, existe mayor dificultad para mantener actualizados los registros y controlar las relaciones entre los datos.

Por esta razón, el proyecto plantea una aplicación web que permita centralizar la gestión de productos y categorías, aplicar validaciones antes de almacenar información y controlar mediante permisos qué operaciones puede realizar cada usuario.

La solución desarrollada busca demostrar la aplicación práctica de conceptos de desarrollo web, incluyendo:

- Arquitectura cliente-servidor.
 - APIs REST.
 - Laravel.
 - React.
 - Eloquent ORM.
 - Autenticación.
 - Roles y permisos.
 - Validación.
 - Manejo de errores HTTP.
 - Rutas protegidas.
 - Relaciones entre tablas.
 - Operaciones CRUD.
-

3. OBJETIVOS

3.1 Objetivo general

Desarrollar un sistema web de gestión de inventario utilizando Laravel y React, que permita administrar categorías y productos mediante operaciones CRUD, aplicando autenticación, autorización, validación de datos, relaciones entre entidades y comunicación mediante una API.

3.2 Objetivos específicos

1. Implementar autenticación para controlar el acceso de los usuarios al sistema.
2. Implementar roles y permisos en Laravel para restringir las operaciones según el tipo de usuario.
3. Desarrollar el CRUD de categorías.
4. Desarrollar el CRUD de productos.
5. Establecer la relación entre categorías y productos mediante Eloquent ORM.
6. Implementar formularios controlados en React conectados con la API de Laravel.
7. Implementar validaciones en el backend y mostrar los errores correspondientes en el frontend.
8. Implementar el manejo de respuestas HTTP como 401, 403, 404 y 422.
9. Implementar una navegación SPA mediante React Router.

10. Implementar búsqueda de productos por nombre o código.
 11. Aplicar medidas básicas de seguridad, evitando exponer credenciales y validando las operaciones importantes desde el backend.
 12. Verificar el funcionamiento del sistema mediante las pruebas establecidas en la guía del proyecto.
-

4. TECNOLOGÍAS UTILIZADAS

Para el desarrollo del sistema se utilizaron las siguientes tecnologías:

Tecnología	Uso
Laravel	Desarrollo del backend y API
PHP	Lenguaje utilizado por Laravel
React	Desarrollo de la interfaz frontend
JavaScript	Lógica del frontend
Vite	Herramienta de desarrollo y construcción del frontend
React Router	Navegación y rutas de la SPA
Tailwind CSS	Estilos de la interfaz React
MySQL	Sistema de gestión de base de datos
Eloquent ORM	Relación y acceso a los datos
Laravel Gates	Autorización mediante permisos
Git	Control de versiones
GitHub	Repositorio del proyecto

5. ARQUITECTURA DEL SISTEMA

El sistema utiliza una arquitectura separada entre frontend y backend.

5.1 Backend

Laravel funciona como servidor de la aplicación y es responsable de:

- Autenticación.
- Autorización.
- Validación.
- Procesamiento de solicitudes.
- Acceso a la base de datos.
- Relaciones entre modelos.
- Respuestas JSON de la API.

Entre los archivos principales utilizados se encuentran:

- `app/Http/Controllers/Api/ProductController.php`
- `app/Http/Controllers/Api/CategoryController.php`
- `app/Models/Product.php`
- `app/Models/Category.php`
- `app/Models/User.php`
- `database/seeder/RolePermissionSeeder.php`
- `database/seeder/DatabaseSeeder.php`

5.2 Frontend

React funciona como una aplicación SPA y se encarga de:

- Mostrar las interfaces.
- Gestionar formularios.
- Navegar entre las diferentes vistas.
- Mostrar estados de carga.
- Mostrar mensajes de error.
- Mostrar resultados de las solicitudes.
- Enviar información a la API.

Entre los archivos principales se encuentran:

- `frontend/src/pages/Products.jsx`
- `frontend/src/pages/ProductCreate.jsx`

- frontend/src/pages/ProductEdit.jsx
- frontend/src/pages/Categories.jsx
- frontend/src/pages/CategoryCreate.jsx
- frontend/src/pages/CategoryEdit.jsx
- frontend/src/services/api.js
- frontend/src/context/AuthContext.jsx
- frontend/src/routes/AppRoutes.jsx

También se utilizan componentes relacionados con el diseño general de la aplicación, como DashboardLayout, Sidebar y ProtectedRoute.

6. ESTRUCTURA DEL PROYECTO

El proyecto está organizado separando el backend Laravel del frontend React.

De manera general, la estructura contiene:

```

app-movil/
|
├── app/
|   ├── Http/
|   |   └── Controllers/
|   |       └── Api/
|   └── Models/
|
├── database/
|   ├── migrations/
|   └── seeders/
|
├── frontend/
|   └── src/
|       ├── context/
|       ├── pages/
|       └── routes/

```

```
|      └─ services/
|
|      └─ resources/
|      └─ routes/
|      └─ .env.example
|      └─ composer.json
|      └─ README.md
```

La aplicación React se ejecuta mediante Vite y se comunica con Laravel mediante los endpoints de la API.

7. AUTENTICACIÓN, ROLES Y PERMISOS

7.1 Autenticación

El sistema dispone de autenticación para evitar que usuarios no autenticados puedan acceder a las áreas privadas de la aplicación.

En el frontend existe un AuthContext encargado de manejar el estado relacionado con el usuario autenticado.

Las rutas privadas se protegen mediante ProtectedRoute.

Cuando se intenta acceder al área privada sin una sesión válida, el usuario es dirigido al proceso de inicio de sesión.

Además, las solicitudes realizadas desde el frontend utilizan:

credentials: 'include'

para permitir el envío de las credenciales de sesión correspondientes.

7.2 Roles

El sistema contempla los roles:

- admin
- worker

El rol admin dispone de los permisos correspondientes a las operaciones administrativas configuradas.

El rol worker dispone de un conjunto limitado de permisos.

7.3 Permisos

Los permisos definidos incluyen operaciones relacionadas con:

productos.ver
productos.crear
productos.editar
productos.eliminar

categorias.ver
categorias.crear
categorias.editar
categorias.eliminar

movimientos.ver
movimientos.crear
movimientos.editar
movimientos.eliminar

La asignación de roles y permisos se encuentra principalmente en:

database/seeder/RolePermissionSeeder.php

El backend utiliza `Gate::authorize()` para comprobar los permisos antes de ejecutar determinadas operaciones.

Por ejemplo, en `ProductController` se comprueba:

```
Gate::authorize('productos.crear');
```

antes de permitir la creación de un producto.

Esto permite que la seguridad no dependa únicamente de la interfaz React.

8. MÓDULO DE CATEGORÍAS

El módulo de categorías permite administrar las categorías utilizadas por los productos.

Se implementaron las operaciones principales:

- Listar categorías.
- Crear categoría.
- Editar categoría.
- Eliminar categoría.

Las vistas principales del frontend son:

Categories.jsx

CategoryCreate.jsx

CategoryEdit.jsx

La comunicación con Laravel se realiza mediante el servicio centralizado:

frontend/src/services/api.js

El controlador correspondiente se encuentra en:

app/Http/Controllers/Api/CategoryController.php

La consulta de categorías se realiza ordenando los registros por nombre.

9. MÓDULO DE PRODUCTOS

El módulo de productos constituye el módulo principal del sistema.

Permite:

- Listar productos.
- Crear productos.
- Editar productos.
- Eliminar productos.
- Asociar productos con categorías.
- Buscar productos por nombre o código.

Las principales páginas React son:

Products.jsx

ProductCreate.jsx

ProductEdit.jsx

El controlador correspondiente es:

app/Http/Controllers/Api/ProductController.php

9.1 Datos del producto

El formulario de productos contempla campos como:

- category_id
- name
- code
- description
- price
- stock
- minimum_stock
- image

9.2 Validación

El backend valida los datos antes de almacenarlos.

Por ejemplo, la categoría debe existir:

`'category_id' => ['required', 'exists:categories,id']`

También se valida que el código sea único durante la creación:

`'code' => ['required', 'string', 'max:255', 'unique:products,code']`

El precio debe ser numérico y no negativo, mientras que los valores de stock deben corresponder a cantidades válidas.

10. RELACIÓN CATEGORÍA-PRODUCTO

Una de las partes principales del proyecto es la relación entre las tablas `categories` y `products`.

10.1 Clave foránea

La tabla de productos utiliza:

`category_id`

como clave foránea que identifica la categoría asociada al producto.

10.2 Relación en Category

El modelo `Category` establece una relación de uno a muchos con los productos:

`Category → hasMany → Product`

Es decir, una categoría puede estar asociada con varios productos.

10.3 Relación en Product

El modelo `Product` establece una relación inversa:

`Product → belongsTo → Category`

Esto significa que cada producto pertenece a una categoría.

10.4 Validación de la relación

Antes de almacenar el producto, Laravel comprueba que el `category_id` corresponda a una categoría existente mediante:

```
exists:categories,id
```

Por lo tanto, el backend evita guardar un producto asociado a una categoría inexistente.

10.5 Consulta de la relación

El controlador de productos utiliza:

```
Product::with('category')
```

para obtener también la información de la categoría asociada.

De esta manera, el frontend no necesita trabajar únicamente con el identificador de la categoría, sino que puede recibir la información relacionada.

11. API Y COMUNICACIÓN REACT-LARAVEL

Para evitar repetir la configuración de las solicitudes HTTP en cada componente, se creó un servicio centralizado:

```
frontend/src/services/api.js
```

Este servicio utiliza como base:

```
/api
```

y proporciona métodos para:

GET

POST

PUT

DELETE

11.1 Solicitud GET

Ejemplo utilizado en el frontend:

```
api.get('/products')
```

11.2 Creación

Para crear un producto:

```
api.post('/products', body)
```

11.3 Actualización

Para actualizar un producto:

```
api.put(`/products/${productId}`, body)
```

11.4 Eliminación

Para eliminar:

```
api.delete(`/products/${productId}`)
```

11.5 Manejo de respuestas

El servicio verifica:

```
if (!response.ok)
```

y obtiene el código HTTP correspondiente.

También conserva:

- mensaje del servidor;
- código de estado;
- errores de validación.

Esto permite que cada componente pueda responder de manera diferente según el tipo de error recibido.

12. FORMULARIOS Y VALIDACIONES

Los formularios de React trabajan con datos controlados mediante estado.

El proceso general consiste en:

1. El usuario introduce los datos.
2. React mantiene los valores del formulario.
3. El usuario ejecuta la acción de guardar.
4. El frontend envía los datos mediante `api.js`.
5. Laravel recibe la solicitud.
6. Laravel valida los datos.
7. Si la validación es correcta, se almacena el registro.
8. Laravel devuelve una respuesta JSON.
9. React procesa la respuesta.
10. La interfaz muestra el resultado correspondiente.

12.1 Errores de validación

Cuando Laravel devuelve un código HTTP 422, el servicio API conserva los errores enviados por el backend.

Esto permite mostrar los errores asociados a los campos correspondientes del formulario.

Por ejemplo, si un producto intenta utilizar una categoría inexistente, Laravel rechaza el registro mediante la regla:

```
exists:categories,id
```

13. MANEJO DE ERRORES HTTP

El sistema contempla diferentes respuestas HTTP y las utiliza para determinar cómo debe reaccionar la interfaz.

13.1 Código 401

El código 401 representa una solicitud realizada sin autenticación válida.

En el proyecto, el acceso a las áreas privadas requiere una sesión válida. Cuando no existe una sesión, el usuario debe pasar nuevamente por el proceso de autenticación.

13.2 Código 403

El código 403 se utiliza cuando el usuario está autenticado, pero no tiene autorización para realizar una operación determinada.

La autorización se comprueba en Laravel mediante `Gate::authorize()`.

Esto evita depender exclusivamente de los controles visuales de React.

13.3 Código 404

El frontend dispone de una ruta para manejar páginas no encontradas.

La configuración de React Router incluye una ruta comodín para mostrar la página correspondiente cuando no existe una ruta definida.

13.4 Código 422

El código 422 se utiliza para errores de validación.

Un ejemplo real comprobado durante el desarrollo fue la prueba de una solicitud de creación de producto utilizando un `category_id` inexistente.

Laravel rechazó la solicitud debido a la regla:

`exists:categories,id`

y devolvió una respuesta de validación.

13.5 Código 500

Los errores inesperados del servidor se manejan como errores generales para evitar presentar información sensible de la aplicación al usuario.

14. SPA, RUTAS Y NAVEGACIÓN

El frontend se desarrolló como una aplicación SPA mediante React Router.

Las rutas principales incluyen:

`/inventory`
`/products`
`/products/create`
`/products/:id/edit`
`/categories`
`/categories/create`
`/categories/:id/edit`

También existe una ruta para las páginas no encontradas.

14.1 Rutas parametrizadas

Las rutas:

`/products/:id/edit`
`/categories/:id/edit`

permiten utilizar el identificador del registro para cargar la información correspondiente.

Por ejemplo:

`/products/ID/edit`

permite acceder al formulario de edición de un producto específico.

14.2 Layout

La aplicación utiliza un layout general compuesto por:

- Sidebar.
- Header.
- Área principal de contenido.

El contenido de las diferentes páginas se presenta mediante el sistema de rutas de React.

14.3 Menú según permisos

El componente Sidebar utiliza el estado de autenticación y la función de comprobación de permisos para mostrar las opciones disponibles.

Esto mejora la experiencia del usuario, aunque la autorización definitiva continúa realizándose en Laravel.

15. BÚSQUEDA DE PRODUCTOS

La guía exige que el módulo principal implemente al menos una funcionalidad entre:

- búsqueda;
- filtros;
- paginación.

En este proyecto se implementó **búsqueda de productos**.

El endpoint de productos recibe un parámetro de búsqueda y permite buscar por:

- nombre;
- código.

En ProductController.php, la consulta utiliza la condición correspondiente para comparar el término recibido con esos campos.

De esta manera, el módulo de productos permite localizar registros sin depender únicamente de recorrer manualmente toda la lista.

16. SEGURIDAD

Durante el desarrollo se aplicaron diferentes medidas de seguridad.

16.1 Contraseñas

Las contraseñas son gestionadas mediante el sistema de autenticación de Laravel y no se almacenan como texto plano en el código fuente.

16.2 Protección de endpoints

Las operaciones importantes se encuentran protegidas por autenticación y autorización.

16.3 Autorización en backend

Las acciones sensibles utilizan comprobaciones como:

```
Gate::authorize(...)
```

Esto significa que un usuario no puede obtener permisos simplemente modificando la interfaz React.

16.4 Validación

Laravel valida los datos antes de almacenarlos.

Entre las validaciones se encuentra la comprobación de claves foráneas mediante:

```
exists:categories,id
```

16.5 Asignación de datos

Los modelos utilizan las propiedades necesarias para controlar los campos que pueden ser asignados mediante Eloquent.

16.6 Variables de entorno

El archivo `.env` no forma parte del repositorio público.

Se mantiene un archivo:

`.env.example`

como referencia para configurar el proyecto.

No deben incluirse en GitHub:

- contraseñas;
- claves privadas;
- tokens;
- credenciales;
- archivos `.env`.

16.7 Construcción del frontend

Se verificó que el frontend pueda realizar una construcción de producción mediante:

`npm run build`

La construcción finalizó correctamente.

17. PRUEBAS DEL SISTEMA

De acuerdo con la guía del proyecto, se realizaron las pruebas P1 a P14.

Las capturas obtenidas durante estas pruebas están en la URL:

<https://docs.google.com/document/d/10loJoZk-TtSzIAD5SD9zlxHNaFZ-ogL7/edit?usp=sharing&oid=11149536439207761249&rtpof=true&sd=true>

19. RESULTADOS Y CUMPLIMIENTO DE LA GUÍA

El desarrollo permitió implementar los principales requerimientos técnicos establecidos en la guía.

Requisito	Estado
Autenticación	Implementado
Logout	Implementado
Rutas protegidas	Implementado
Roles	Implementado
Permisos backend	Implementado
CRUD de categorías	Implementado
CRUD de productos	Implementado
Relación categoría-producto	Implementado
Clave foránea	Implementada
Validación backend	Implementada
Errores de validación 422	Implementados
Servicio HTTP centralizado	Implementado
Manejo de 401/403/404/422	Implementado

Requisito	Estado
Confirmación antes de eliminar	Implementada
Estados de carga/error/éxito	Implementados
Búsqueda de productos	Implementada
Rutas parametrizadas	Implementadas
Página 404	Implementada
Construcción de producción	Verificada
Repositorio Git	Implementado
Exclusión de archivos sensibles	Verificada
Pruebas P1–P14	Realizadas

El sistema no incorpora un módulo frontend de administración de usuarios, ya que esta funcionalidad no forma parte de la implementación realizada y no se presenta como una característica terminada del proyecto.

20. RESPUESTAS A LAS SEIS PREGUNTAS OBLIGATORIAS

Pregunta 1. Explique el recorrido completo de los datos cuando un usuario guarda un registro: desde el formulario React, pasando por la API, hasta la base de datos y la respuesta que vuelve a la interfaz.

Para explicar el recorrido se puede utilizar como ejemplo la creación de un producto.

El proceso comienza en ProductCreate.jsx, donde el usuario introduce los datos del producto mediante un formulario controlado por React.

Cuando el usuario ejecuta la acción de guardar, el componente utiliza el servicio centralizado:

```
api.post('/products', body)
```

El archivo `frontend/src/services/api.js` construye la solicitud utilizando la base `/api`, envía las credenciales de sesión, gestiona el mecanismo CSRF para las solicitudes que modifican datos y envía el cuerpo como JSON.

La solicitud llega al endpoint correspondiente de Laravel y es procesada por:

```
app/Http/Controllers/Api/ProductController.php
```

Antes de almacenar el producto, el controlador comprueba el permiso:

```
Gate::authorize('productos.crear');
```

Posteriormente Laravel valida los datos recibidos. Entre las validaciones se encuentra:

```
'category_id' => ['required', 'exists:categories,id']
```

Esto comprueba que la categoría seleccionada exista.

También se valida el código del producto y los demás campos definidos por el controlador.

Cuando la validación es correcta, se ejecuta:

```
Product::create($validated);
```

Eloquent guarda el registro en la base de datos.

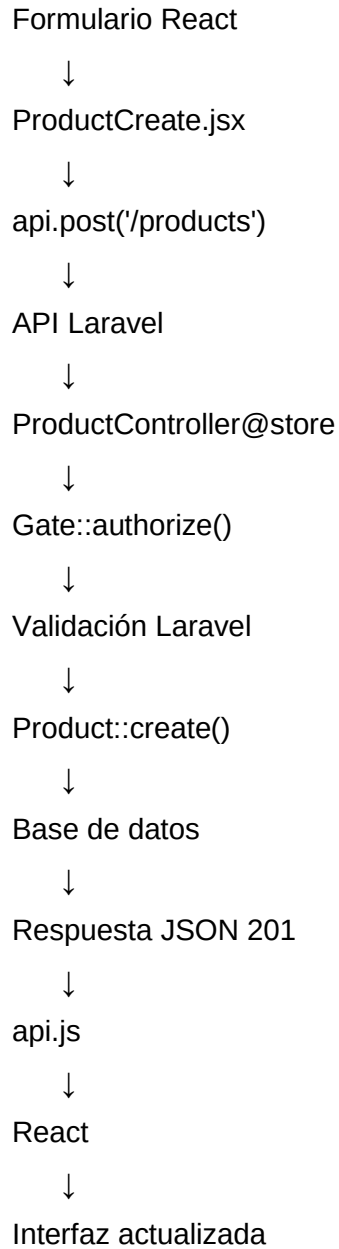
Después, el controlador carga la categoría relacionada:

```
$product->load('category')
```

y devuelve el producto como respuesta JSON, utilizando el código HTTP 201 para indicar que el registro fue creado.

Finalmente, `api.js` recibe la respuesta y devuelve los datos al componente React. `ProductCreate.jsx` puede utilizar esa respuesta para actualizar el estado de la interfaz, mostrar el resultado correspondiente o realizar la navegación definida después de guardar.

Por lo tanto, el recorrido puede resumirse de la siguiente manera:



Pregunta 2. ¿Cómo se implementó la autenticación y cómo se mantiene el acceso a las rutas privadas?

La autenticación se implementa utilizando Laravel como parte responsable de la sesión y del control de acceso.

En React existe AuthContext.jsx, que permite manejar el estado del usuario autenticado dentro de la aplicación.

Las rutas privadas se encuentran protegidas mediante ProtectedRoute. Esto permite que una persona que no tenga una sesión válida no pueda acceder directamente a las páginas privadas de la SPA.

Además, las solicitudes realizadas por el servicio API utilizan:

credentials: 'include'

para incluir las credenciales de sesión en las solicitudes.

El frontend, por tanto, controla la experiencia de navegación, pero la seguridad real no depende solamente de React. Laravel continúa siendo la autoridad del lado del servidor.

Esto es importante porque una persona podría intentar realizar una solicitud directamente contra la API sin utilizar la interfaz React. Por esa razón, los endpoints importantes también están protegidos desde el backend.

Pregunta 3. ¿Cómo funcionan los roles y permisos? ¿Dónde se comprueba realmente una acción restringida y por qué no es suficiente ocultar el botón en React?

El proyecto utiliza roles y permisos para controlar las operaciones disponibles para los usuarios.

Los roles definidos incluyen:

admin
worker

Los permisos se configuran en:

database/seeder/RolePermissionSeeder.php

Entre ellos se encuentran:

productos.ver
productos.crear
productos.editar

productos.eliminar

categorias.ver

categorias.crear

categorias.editar

categorias.eliminar

En React, el componente Sidebar utiliza la información del usuario y la función de comprobación de permisos para mostrar u ocultar determinadas opciones de navegación.

Sin embargo, ocultar un botón no constituye una medida de seguridad.

La comprobación real se realiza en Laravel.

Por ejemplo, para crear un producto, ProductController utiliza:

```
Gate::authorize('productos.crear');
```

Esto significa que aunque un usuario consiguiera enviar manualmente una solicitud HTTP al endpoint de creación, Laravel volvería a comprobar el permiso.

Por esta razón, React se utiliza para controlar la experiencia de usuario, mientras que Laravel realiza la autorización efectiva.

Pregunta 4. Describa la relación entre dos tablas del proyecto, indicando la clave foránea y las relaciones Eloquent.

Las dos tablas principales utilizadas para explicar la relación son:

categories

products

Un producto pertenece a una categoría, mientras que una categoría puede tener varios productos.

En la tabla products se utiliza:

`category_id`

como clave foránea.

En el modelo Category se establece la relación:

`hasMany(Product)`

Esto representa que una categoría puede tener muchos productos.

En el modelo Product se establece:

`belongsTo(Category)`

Esto representa que un producto pertenece a una categoría.

La relación también se protege mediante validación del backend:

`'exists:categories,id'`

Además, el controlador de productos utiliza:

`Product::with('category')`

para obtener la información de la categoría relacionada al consultar los productos.

La estructura conceptual es:

Category

|

| 1

|

|

| N



Product

Por lo tanto:

categories.id

↑

|

products.category_id

Pregunta 5. Explique cómo se manejan los códigos 401, 403, 404 y 422 y proporcione al menos un ejemplo real del proyecto.

401 — No autenticado

El código 401 representa una solicitud que requiere autenticación.

En el proyecto, las áreas privadas requieren una sesión válida. Durante las pruebas se comprobó el comportamiento de acceso al área privada sin una sesión, produciéndose el redireccionamiento hacia el proceso de autenticación.

403 — No autorizado

El código 403 corresponde a una situación en la que el usuario está autenticado pero no dispone del permiso necesario para ejecutar una operación.

El proyecto utiliza `Gate::authorize()` en Laravel para realizar estas comprobaciones.

Por ejemplo:

```
Gate::authorize('productos.crear');
```

impide que un usuario sin el permiso correspondiente pueda crear productos.

404 — Recurso o ruta no encontrada

React Router dispone de una ruta para las páginas no existentes.

Esto permite mostrar una respuesta de página no encontrada cuando el usuario intenta acceder a una ruta de la SPA que no está definida.

422 — Error de validación

El código 422 se utiliza cuando los datos enviados no cumplen las reglas de validación.

Un ejemplo real del proyecto fue la prueba de creación de un producto utilizando un `category_id` inexistente.

La regla:

```
'exists:categories,id'
```

impidió que el registro pudiera almacenarse y Laravel devolvió una respuesta 422.

De esta manera, el sistema diferencia entre:

401 → falta autenticación

403 → falta autorización

404 → recurso/ruta no encontrada

422 → datos enviados no válidos

Pregunta 6. Describa un problema técnico importante, cómo se identificó la causa y cuál fue la solución aplicada.

Uno de los problemas técnicos importantes durante el desarrollo ocurrió en la configuración de estilos de Tailwind CSS dentro del frontend React.

Problema

Inicialmente, los estilos de Tailwind no se aplicaban correctamente en la aplicación React y se produjo un error relacionado con las directivas utilizadas en el archivo CSS.

Uno de los mensajes observados estaba relacionado con:

```
@layer base
```

y la ausencia de la configuración/directiva correspondiente de Tailwind.

Identificación de la causa

Para identificar el problema se revisó la configuración del frontend y, específicamente, el archivo:

frontend/src/index.css

También se revisó la configuración utilizada por Vite y Tailwind.

El problema estaba relacionado con que la forma en que se estaban utilizando las directivas CSS no correspondía con la configuración de Tailwind utilizada en el frontend.

Solución

Se modificó el inicio de:

frontend/src/index.css

para utilizar:

```
@import "tailwindcss";
```

También se conservó la configuración de Tailwind mediante el plugin correspondiente de Vite.

Antes de realizar el cambio se creó una copia de respaldo del archivo CSS.

Resultado

Después de aplicar el cambio, Tailwind comenzó a aplicarse correctamente en la interfaz React.

Esto permitió continuar con el desarrollo de las rutas, formularios, CRUD y demás funcionalidades sin tener que modificar el archivo CSS de Laravel destinado a sus propias vistas.

21. PROBLEMAS TÉCNICOS Y LECCIONES APRENDIDAS

Durante el desarrollo se presentaron diferentes situaciones que requirieron revisión y pruebas.

Entre ellas se encuentra el problema inicial de configuración de Tailwind en React, que fue solucionado modificando la forma de importación de Tailwind en index.css.

También se presentó durante una prueba manual de la API una respuesta 419 relacionada con CSRF. Esto permitió comprobar que las solicitudes que modifican información deben realizarse respetando el mecanismo de protección CSRF configurado.

Posteriormente, utilizando el flujo adecuado, se pudo realizar la prueba de validación de una clave foránea inexistente y obtener la respuesta 422 esperada.

Estas situaciones permitieron comprobar que el funcionamiento de una aplicación SPA no depende únicamente de la interfaz, sino también de la correcta comunicación entre React, la API, la autenticación, la protección CSRF, las validaciones y la base de datos.

22. CONCLUSIONES

El desarrollo del Sistema Web de Gestión de Inventario permitió integrar diferentes tecnologías y conceptos de desarrollo web en una aplicación funcional.

Se implementó un backend Laravel encargado de la autenticación, autorización, validación, procesamiento de solicitudes y comunicación con la base de datos.

Por otro lado, React permitió construir una interfaz SPA organizada mediante componentes, rutas, formularios y un servicio centralizado de comunicación con la API.

Uno de los aspectos principales del proyecto fue la implementación de la relación entre categorías y productos. Esta relación se encuentra respaldada tanto por la estructura de la base de datos como por las relaciones de Eloquent y las validaciones realizadas en Laravel.

También se implementaron roles y permisos, de manera que las restricciones importantes no dependan únicamente de los elementos visuales del frontend.

Las pruebas realizadas permitieron verificar diferentes escenarios, incluyendo autenticación, acceso a rutas privadas, permisos, creación, validación, relaciones, edición, eliminación, búsqueda y navegación SPA.

Finalmente, el proyecto permitió aplicar de forma práctica conceptos relacionados con Laravel, React, APIs REST, ORM, autenticación, autorización, validación, manejo de errores HTTP, Git y desarrollo de aplicaciones web.

23. TRABAJO FUTURO

Las siguientes funcionalidades pueden considerarse como posibilidades de ampliación del sistema, pero **no forman parte de la implementación presentada como terminada en este informe**:

- Implementar un módulo completo de administración de usuarios desde React.
- Ampliar la gestión de movimientos de inventario.
- Incorporar funcionalidades adicionales de consulta y reportes.
- Incorporar paginación si el volumen de productos lo requiere.
- Incorporar nuevas funcionalidades según las necesidades del negocio.

Estas posibilidades se presentan como trabajo futuro y no como características actualmente terminadas.

24. REPOSITORIO DEL PROYECTO

El código fuente del proyecto se encuentra en GitHub:

Repositorio:

<https://github.com/Cesarza12/App-Inventory.git>

.

25. DOCUMENTACIÓN Y ENTREGA

Como parte de la entrega final, el proyecto debe incluir la documentación correspondiente.

La estructura recomendada para la documentación final es:

documentacion/

└─ Informe_Proyecto_Final.pdf

El informe PDF deberá contener esta documentación junto con las evidencias de las pruebas P1–P14

El archivo README.md del proyecto debe contener las instrucciones necesarias para:

1. Obtener el proyecto.
2. Instalar las dependencias de Laravel.
3. Configurar .env.
4. Ejecutar las migraciones.
5. Ejecutar los seeders correspondientes.
6. Instalar las dependencias del frontend.
7. Ejecutar Laravel.
8. Ejecutar React/Vite.
9. Conocer los módulos principales.
10. Acceder a la documentación final.

26. ANEXOS Y EVIDENCIAS

Url:

<https://docs.google.com/document/d/10loJoZk-TtSzIAD5SD9zIxHNaFZ-ogL7/edit?usp=sharing&oid=11149536439207761249&rtpof=true&sd=true>

27. CIERRE

El Sistema Web de Gestión de Inventario integra un backend desarrollado con Laravel y un frontend desarrollado con React, estableciendo una comunicación mediante API y aplicando mecanismos de autenticación, autorización, validación y manejo de errores.

La implementación realizada cumple con los componentes principales establecidos en la guía del proyecto: autenticación, roles y permisos, CRUD de categorías, CRUD de productos, relación entre entidades, formularios conectados a la API, validaciones, manejo de respuestas HTTP, rutas SPA, búsqueda de productos, control de seguridad, pruebas funcionales y control de versiones mediante Git.